

Democratic and Popular Republic of Algeria
Ministry of Higher Education and Scientific Research
Higher School of Computer Science, May 8, 1945, Sidi Bel Abbes

Academic year: 2025-2026
2 SC: IASD

Semester 2
Module: Big Data Technologies



Chapter 1: Hadoop



BIG DATA: Introduction

- The concept of Big Data became popular in 2012 to describe the fact that companies are facing increasingly large volumes of data to process, which present significant commercial and marketing challenges.



BIG DATA Technologies: Definition

- These Big Data have become difficult to work with using traditional database management tools.
- BIG DATA technologies: This refers to a set of technologies, architectures, tools, and procedures that allow an organization to quickly capture and process large, heterogeneous, and constantly changing quantities of data, and to extract relevant information at an affordable cost.

BIG DATA: Characteristics (5V)

- **Volume:** The continuous growth of data to manage of all types.
- **Variety:** Data in multiple formats (texts, sounds, images...).
- **Velocity:** Data must be stored upon arrival; otherwise, there is a risk of losing information.
- **Veracity:** Refers to the quality, reliability, and trustworthiness of the data.
- **Value:** The success of a Big Data project is only meaningful to users if it brings added value and new insights.



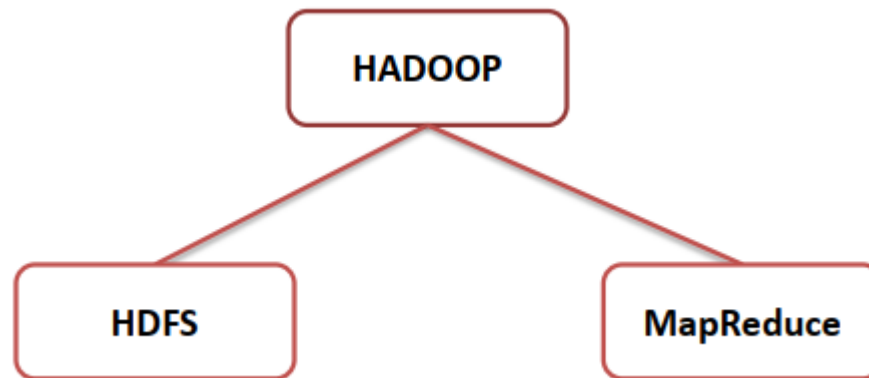


Hadoop: Presentation

- Apache Hadoop is an open-source framework designed to perform distributed processing on massive volumes of data.
- It addresses all issues related to distributed computing, such as data access and sharing, fault tolerance, and task distribution among cluster machines.
- The programmer only needs to focus on software development for task execution.

Hadoop: Presentation

- Hadoop consists of two major components:
 - **Hadoop Distributed File System (HDFS):** Designed for the distributed storage of data.
 - **Distributed Programming Framework - MapReduce:** Designed for the distributed processing of data.



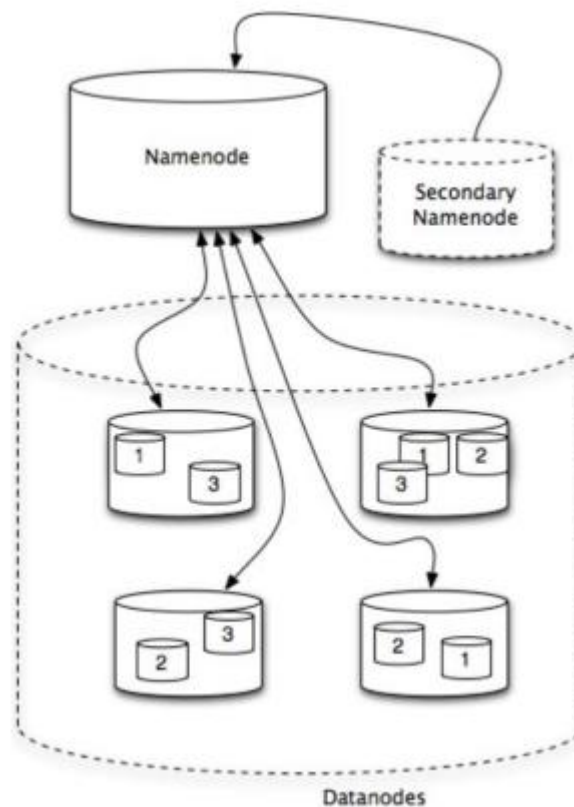
HDFS

HDFS: Architecture

- An HDFS machine architecture (also known as an HDFS cluster) relies on three major components:
 - **NameNode:** A central service (also called the master) responsible for managing the state of the file system. It maintains the file system's directory tree and metadata for all the files and directories in a Hadoop system. The NameNode is aware of the DataNodes where the blocks are stored.
 - **Secondary Namenode:** The **Secondary NameNode** improves performance and reliability by **periodically checkpointing metadata**, preventing large edits logs and reducing NameNode restart time.

HDFS: Architecture

- **Datanode:** Contains the data blocks. The Datanodes are under the command of the Namenode and are referred to as **Workers**. They are therefore engaged by the Namenode during read and write operations.



HDFS: Architecture

1. How the NameNode manages metadata

The **NameNode** is the master of HDFS. It does **not store actual data**, but it stores **metadata**, such as:

- File names and directory structure
- File permissions
- Mapping of files to data blocks and DataNodes

HDFS: Architecture

To manage this metadata efficiently, the NameNode uses **two files**:

a) fsimage

- A **snapshot** of the entire HDFS metadata at a specific point in time
- Represents the *current state* of the file system when it was last saved

It includes:

- Directory structure
- File names
- Permissions and ownership
- Mapping of files → blocks

HDFS: Architecture

b) edits (EditLogs)

- A **transaction log** that records every change made after the last **fsimage**
- Examples of edits:
 - Create a file
 - Delete a directory
 - Change permissions
 - Move or rename a file

HDFS: Architecture

2. What happens when the NameNode starts?

When the NameNode starts or restarts, it follows these steps:

1. Reads the **fsimage** file
2. Applies all operations stored in the **edits** log
3. Reconstructs the latest HDFS state in memory
4. Writes a **new fsimage**
5. Starts normal operation with a **new (empty) edits file**

HDFS: Architecture

Example

Suppose:

- fsimage was created at **10:00 AM**
- After that, users:
 - Created 10,000 files
 - Deleted 2,000 files
 - Modified permissions on many files

All these changes are written to the **edits** file.

If the NameNode restarts at **6:00 PM**, it must replay **all those edits**, which can take a long time.

HDFS: Architecture

3. The problem with large edits files

The NameNode **only merges fsimage and edits during startup**. On a busy cluster:

- The **edits file keeps growing**
- It consumes more disk space and memory
- NameNode restart becomes **slow**

HDFS: Architecture

Example

In a large production cluster:

- Millions of file operations happen every hour
- The edits file can grow to **several gigabytes**
- A NameNode restart may take **many minutes or even hours**

This is where the **Secondary NameNode** becomes important.

HDFS: Architecture

4. Role of the Secondary NameNode

The **Secondary NameNode** is a helper daemon whose main job is to **perform periodic checkpointing**.

What is checkpointing?

Checkpointing means:

- Merging **fsimage + edits**
- Creating a **new, updated fsimage**
- Allowing the NameNode to **truncate the edits file**

HDFS: Architecture

How the Secondary NameNode works (step-by-step)

1. Secondary NameNode requests the current **fsimage** and **edits** from the NameNode
2. It merges them
3. Creates a **new checkpointed fsimage**
4. Sends the new fsimage back to the NameNode
5. NameNode replaces the old fsimage and starts a fresh edits log

HDFS: Architecture

5. Example of checkpointing in action

Assume:

- fsimage was last checkpointed at **9:00 AM**
- By **10:00 AM**, the edits file contains **500,000 changes**

At **10:00 AM**:

- Secondary NameNode starts checkpointing
- Merges fsimage + 500,000 edits
- Creates a new fsimage
- NameNode clears the old edits file

Result: - Edits file size becomes small again

- NameNode restart is now fast

HDFS: Architecture

6. Why Secondary NameNode runs on a separate machine

Checkpointing requires:

- Loading **entire HDFS metadata into memory**
- Significant CPU and disk I/O

Therefore:

- Secondary NameNode is usually deployed on a **separate machine**
- Its memory requirements are **similar to the NameNode**
- This prevents performance impact on the NameNode

HDFS: Architecture

7. Checkpoint configuration parameters

Checkpointing is controlled by **two Hadoop configuration settings**:

1) **dfs.namenode.checkpoint.period**

- Default value: **1 hour**
- Defines the **maximum time** between checkpoints

Example: Even if few changes occur, a checkpoint will happen at least once every hour.

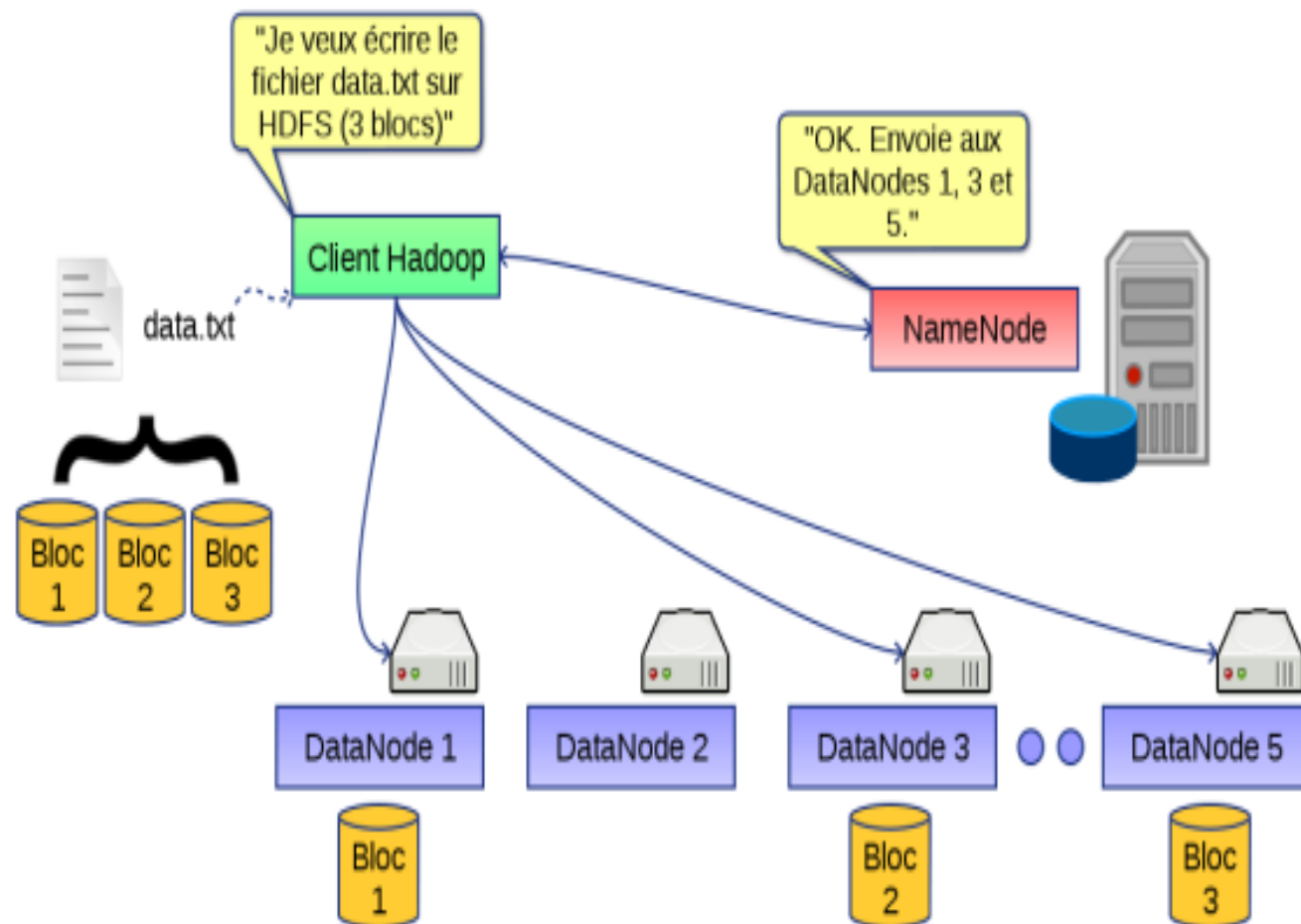
2) **dfs.namenode.checkpoint.txns**

- Default value: **1,000,000 transactions**
- Triggers checkpoint if too many edits accumulate

Example: If 1 million edits happen in 10 minutes, checkpointing starts immediately — no need to wait for 1 hour.

HDFS: Writing a file

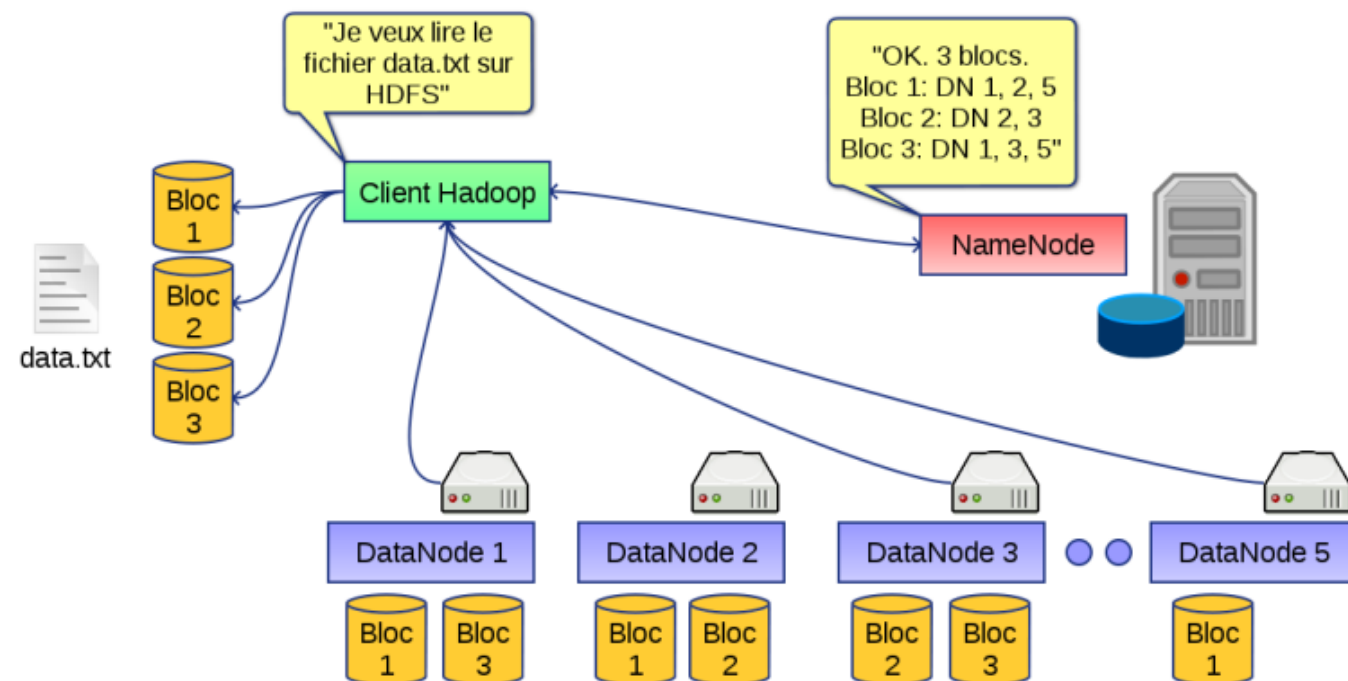
Ecriture HDFS



- Le client indique au NameNode qu'il souhaite écrire un bloc.
- Celui-ci lui indique le DataNode à contacter.
- Le client envoie le bloc au Datanode.
- Les DataNodes répliquent le bloc entre eux.
- Le cycle se répète pour le bloc suivant.

HDFS: Reading a File

Lecture HDFS



- Le client indique au NameNode qu'il souhaite lire un fichier.
- Celui-ci lui indique sa taille et les différents DataNode contenant les N blocs.
- Le client récupère chacun des blocs à un des DataNodes.
- Si un DataNode est indisponible, le client le demande à un autre.

HDFS: The `hadoop fs` Command

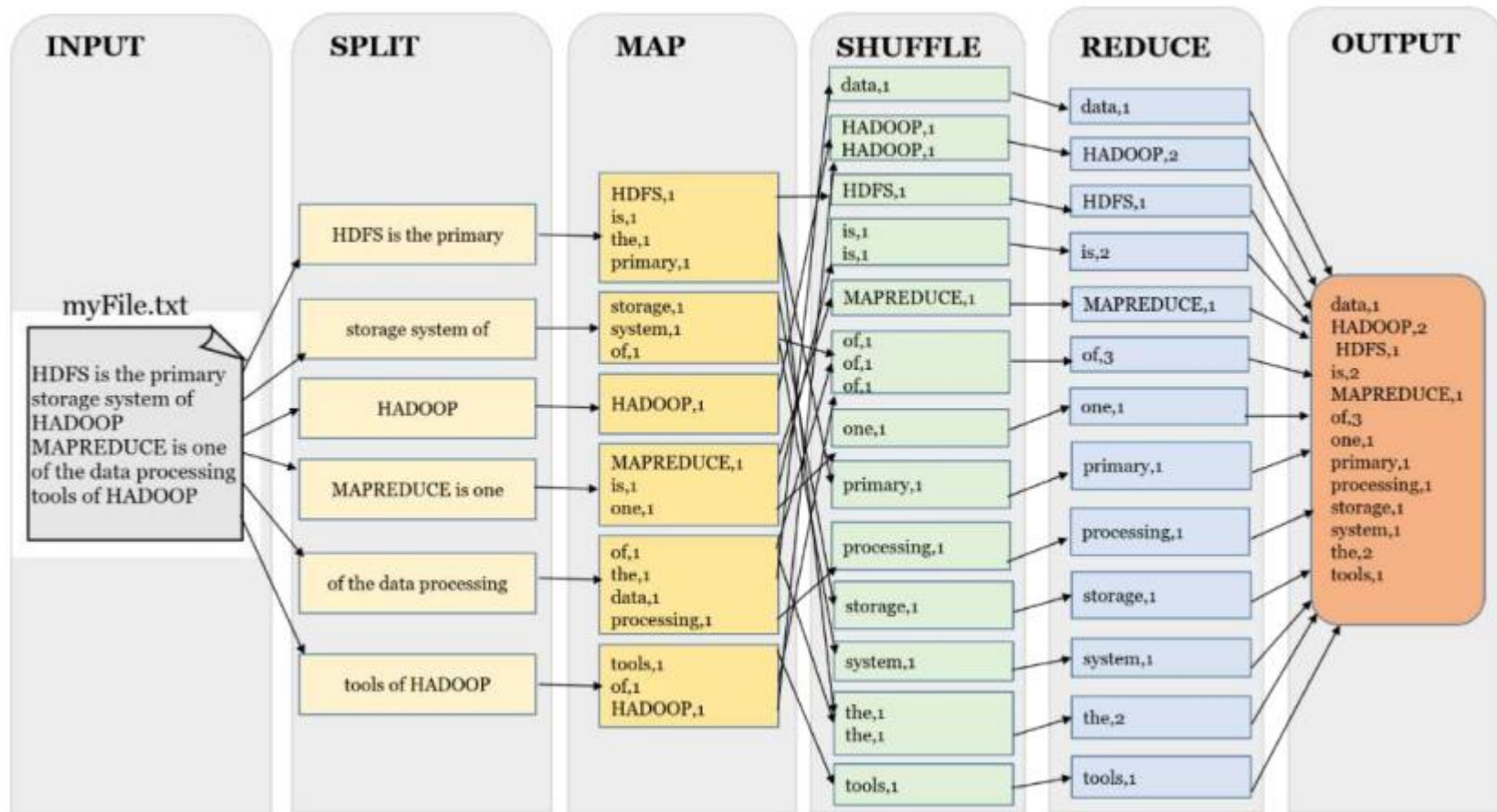
- `hadoop fs -put book.txt /data_input/book.txt`
 - To store the file **book.txt** on HDFS in the directory **/data_input**.
- `hadoop fs -get /data_input/book.txt book.txt`
 - To retrieve the file **/data_input/book.txt** from HDFS and store it in the local file **book.txt**.
- `hadoop fs -mkdir /data_input`
 - To create the directory **/data_input**
- `hadoop fs -rm /data_input/book.txt`
 - To delete the file **/data_input/book.txt**

MapReduce

MapReduce: Presentation

- There are four distinct stages in a MapReduce process:
 1. **Split:** the input data into multiple fragments.
 2. **Map:** each of these fragments to obtain key-value pairs.
 3. **Shuffle:** group these key-value pairs by key.
 4. **Reduce:** the grouped key-indexed data into a final form, with a value for each distinct key.

MapReduce: Presentation



Hadoop Architecture

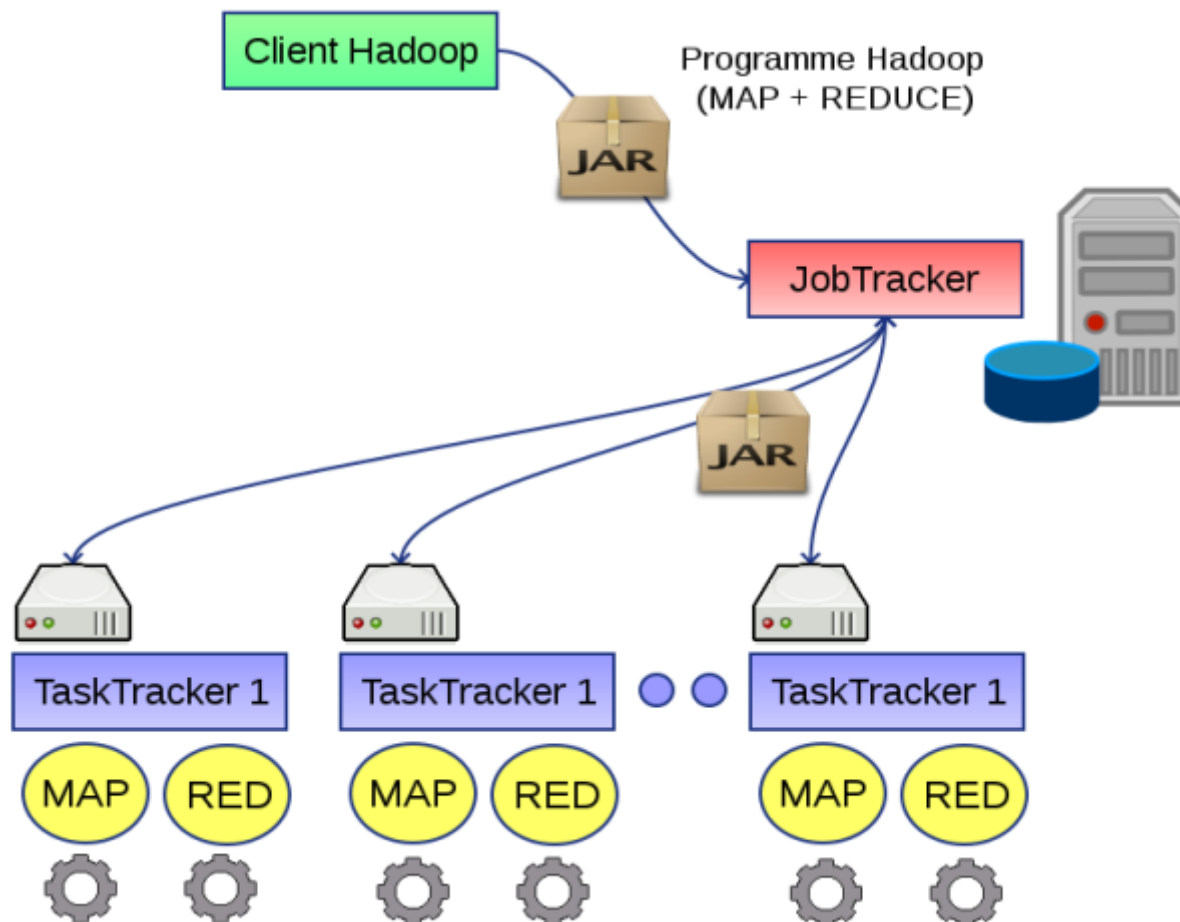
Hadoop Architecture: Presentation

- There are two distinct architectures:
 - The first, consisting of the JobTracker and TaskTracker, corresponds to the first version of the MapReduce execution engine ("MRv1," Hadoop 1.x).
 - Starting from version 2.x, Hadoop integrates a new execution engine: YARN ("MRv2").

MRv1: Presentation

- The task management in Hadoop was based on two servers up until version 2:
 - The **JobTracker**, which directly receives the task to be executed (e.g., a Java .jar file), as well as the input data (names of files stored on HDFS) and the directory where the output data should be stored (also on HDFS). There is only one JobTracker on a single machine in the Hadoop cluster. The JobTracker communicates with the HDFS NameNode and therefore knows where the data is located.
 - The **TaskTracker**, which is in constant communication with the JobTracker and receives the simple operations to perform (MAP/REDUCE) along with the corresponding data blocks (stored on HDFS). There is one TaskTracker on each machine in the cluster.

MRv1 : Presentation



MRv2: YARN

- YARN (Yet Another Resource Negotiator) is a mechanism in Hadoop that allows the management of jobs on a cluster.
- YARN enables users to run MapReduce jobs on data stored in HDFS, monitor their progress, and retrieve the logs generated by the programs.
- In fact, YARN is transparent to the user. When you launch the execution of a MapReduce program, YARN ensures that it is executed as quickly as possible.

MRv2: YARN

- The **JobTracker** was split into two main components:
 - A **resource management module**, called the **Resource Manager (RM)**
 - An **application-specific processing module**, called the **Application Master (AM)**
- The **Resource Manager** coordinates resource usage across the cluster. On each worker node, a daemon called the **Node Manager (NM)** is responsible for managing local resources and executing tasks inside containers.
- The execution of MapReduce (or other processing frameworks) is handled by a **dedicated Application Master**, which negotiates resources with the Resource Manager and coordinates task execution on the Node Managers.

MRv2: YARN

- The **Resource Manager** itself consists of two internal components:
 - A **Scheduler**, responsible for allocating resources (containers) to applications based on scheduling policies.
 - An **Application Manager**, which accepts application submissions from clients and launches the corresponding Application Master.
- The **Resource Manager** allocates resources in the form of **containers**, which represent a bundle of resources (CPU, memory) on a node. Application tasks are executed inside these containers across the cluster.

Hadoop Programming

- Hadoop is developed in Java.
- As a result, MAP/REDUCE tasks are typically implementable in Java.
- A Hadoop program is compiled into a .jar file.
- To develop a Hadoop program, we create three distinct classes:
 - A "Driver" class that contains the **main** function of the program. This class will inform Hadoop of the key/value data types used, the classes responsible for the MAP and REDUCE operations, and the HDFS files to be used for input/output.
 - A MAP class.
 - A REDUCE class.

Thank you for your attention
